Pez Babel con LLM STT TTS

Tiempo estimado necesario: 90 minutos

Introducción

Bienvenido a este proyecto guiado sobre la creación de un Pez Babel. El Pez Babel es una metáfora para un servicio de traducción o una herramienta, basada en una criatura ficticia de la serie "La guía del autoestopista galáctico" de Douglas Adams.

En este proyecto, crearás un asistente de traducción por voz utilizando Watsonx y las bibliotecas de IBM Watson Speech para Embed. El proyecto guiado te llevará a construir un asistente de traducción que puede recibir entrada de voz, convertirla a texto utilizando tecnología de reconocimiento de voz, enviar el texto al modelo mistralai/mistral-large de Watsonx, recibir una respuesta en otros idiomas, convertirla a voz utilizando tecnología de texto a voz y finalmente reproducirla al usuario. El traductor de voz tendrá un frontend responsivo utilizando HTML, CSS y JavaScript, y un backend confiable utilizando Flask.

Haz clic aquí para jugar con una demostración de la aplicación final que crearás.

Al final del curso, tendrás las habilidades para crear tu propio asistente de traducción impulsado por IA capaz de recibir entrada de voz y proporcionar traducciones en varios idiomas. También tendrás una base sólida en desarrollo web utilizando Python, Flask, HTML, CSS y JavaScript, junto con una aplicación de pila completa totalmente funcional que seguramente impresionará a cualquiera que interactúe con ella.

Antes de comenzar, proporcionemos un poco de contexto sobre cada tema.

Watsonx

The content of this lab is licensed under Apache 2.0

confiables en toda tu empresa.

Los componentes centrales incluyen un estudio para nuevos modelos fundamentales, IA generativa y aprendizaje automático; un almacén de datos adecuado construido sobre una arquitectura de lago de datos abierto; y un conjunto de herramientas, para acelerar flujos de trabajo de IA que se construyen con responsabilidad, transparencia y explicabilidad.

Los asistentes de IA de Watsonx empoderan a las personas en tu organización para realizar trabajos sin conocimientos especializados en una variedad de procesos y aplicaciones empresariales, incluyendo la automatización del servicio al cliente, la generación de código y la automatización de flujos de trabajo clave en departamentos como RRHH.

Bibliotecas de IBM Watson Speech para Embed

IBM Watson® Speech Libraries for Embed es un conjunto de bibliotecas de texto a voz y de voz a texto en contenedores diseñadas para ofrecer a nuestros socios de IBM una mayor flexibilidad para incorporar lo mejor de la tecnología de IBM Research® en sus soluciones. Ahora disponibles como IA embebible, los socios obtienen mayores capacidades para construir aplicaciones de transcripción de voz y síntesis de voz más rápidamente y desplegarlas en cualquier entorno híbrido de múltiples nubes. Estas tecnologías permiten que el asistente se comunique con los usuarios a través de entrada y salida de voz.

Asistentes de voz

Un asistente virtual es un programa diseñado para simular la conversación con usuarios humanos, especialmente a través de Internet utilizando la voz humana natural. Los asistentes pueden ser utilizados en una variedad de industrias, incluyendo el servicio al cliente, el comercio electrónico y la educación.

Python (Flask)

Python es un lenguaje de programación popular que se utiliza ampliamente en el desarrollo web y la ciencia de datos. Flask es un marco web para Python que facilita la construcción de aplicaciones web. Utilizaremos Python y Flask para construir el backend de nuestro asistente de voz. Python es un lenguaje poderoso que es fácil de aprender y tiene un gran ecosistema de bibliotecas y marcos que se pueden aprovechar en proyectos como el nuestro.

The content of this lab is licensed under Apache 2.0

HTML - CSS - JavaScript

HTML (Lenguaje de Marcado de Hipertexto) es un lenguaje de marcado utilizado para estructurar contenido en la web. CSS (Hojas de Estilo en Cascada) es un lenguaje de hojas de estilo utilizado para describir la apariencia y el formato de un documento escrito en HTML. JavaScript es un lenguaje de programación que se utiliza comúnmente para agregar interactividad a las páginas web. Juntas, estas tecnologías nos permiten construir un frontend visualmente atractivo e interactivo para nuestro asistente. Los usuarios podrán interactuar con el asistente de voz a través de una interfaz web construida utilizando HTML, CSS y JavaScript.

Objetivos de aprendizaje

Al final de este proyecto, serás capaz de:

- Explicar los fundamentos de los asistentes de voz y sus diversas aplicaciones.
- Configurar un entorno de desarrollo para construir un asistente utilizando Python, Flask, HTML, CSS y JavaScript.
- Implementar la funcionalidad de voz a texto para permitir que el asistente entienda la entrada de voz de los usuarios.
- Integrar el asistente con el modelo mistralai/mistral-large de Watsonx para dotarlo de un alto nivel de inteligencia y la capacidad de entender y responder a las solicitudes de los usuarios.
- Implementar la funcionalidad de texto a voz para permitir que el asistente se comunique con los usuarios a través de salida de voz.
- Combinar todos los componentes anteriores para crear un asistente funcional que pueda recibir entrada de voz y proporcionar una respuesta hablada.
- Desplegar como aplicación web ejecutando un servidor en modo de desarrollo.

Requisitos previos

Para trabajar en este proyecto, deberías tener un conocimiento fundamental de Python. Además, tener conocimientos básicos de HTML, CSS y JavaScript es recomendable, pero no esencial. Haremos nuestro mejor esfuerzo para explicar cada paso del proceso, así como cualquier código mostrado a lo largo del camino.

The content of this lab is licensed under Apache 2.0

The content of this lab is licensed under Apache 2.0

Paso 1: Comprendiendo la interfaz

En este proyecto, el objetivo es crear una interfaz que permita la comunicación con un asistente de voz, y un backend para gestionar el envío y recepción de respuestas.

El frontend utilizará HTML, CSS y JavaScript con bibliotecas populares como Bootstrap para el estilo básico, Font Awesome para los íconos y JQuery para el manejo eficiente de acciones. La interfaz de usuario será similar a otras aplicaciones de asistentes de voz, como Google Assistant. El código para la interfaz se proporciona y el enfoque del curso está en construir el asistente de voz e integrarlo con varios servicios y APIs. El código proporcionado te ayudará a entender cómo interactúan el frontend y el backend, y a medida que lo revises, aprenderás sobre las partes importantes y cómo funciona, dándote una buena comprensión de cómo opera el frontend y cómo crear esta simple página web.

Primero, configuremos el entorno ejecutando el siguiente código:

bash

```
python3.11 -m venv my_env
source my_env/bin/activate # activate my_env
```

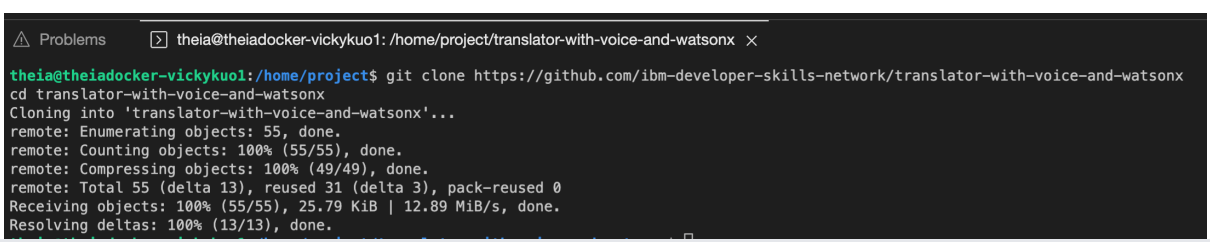
Run

Ejecuta los siguientes comandos en la terminal para recibir el esquema del proyecto, renombrarlo con otro nombre y finalmente moverte a ese directorio:

bash

```
git clone https://github.com/ibm-developer-skills-network/translator-with-
cd translator-with-voice-and-watsonx
```

Run



The content of this lab is licensed under Apache 2.0

instalando los requisitos para el proyecto

```bash
pip install -r requirements.txt
```

Run

Toma una taza de café, tomará de 5 a 10 minutos instalar los requisitos (Puedes continuar con este proyecto mientras se instalan los requisitos).

```plaintext
     )  (
    (   ) )
     ) ( (
   _______)_
 .-'---------|
( C|/\/\/\/\/|
 '-./\/\/\/\/|
   '_________'
    '-------'
```

La siguiente sección ofrece una breve comprensión de cómo funciona el frontend.

HTML, CSS y JavaScript

El archivo `index.html` es responsable del diseño y la estructura de la interfaz web. Este archivo contiene el código para incorporar bibliotecas externas como JQuery, Bootstrap y FontAwesome Icons, así como el CSS (`style.css`) y el código JavaScript (`script.js`) que controlan el estilo y la interactividad de la interfaz.

El archivo `style.css` se encarga de personalizar la apariencia visual de los componentes de la página. También maneja la animación de carga utilizando keyframes de CSS. Los keyframes son una forma de definir los valores de una animación en varios momentos del tiempo, lo que permite una transición suave entre diferentes estilos y crea animaciones dinámicas.

El archivo `script.js` es responsable de la interactividad y funcionalidad de la página. Contiene la mayor parte del código y maneja todas las funciones necesarias, como cambiar entre el modo claro y oscuro, enviar mensajes y mostrar nuevos mensajes en la pantalla. Incluso permite a los usuarios grabar audio.

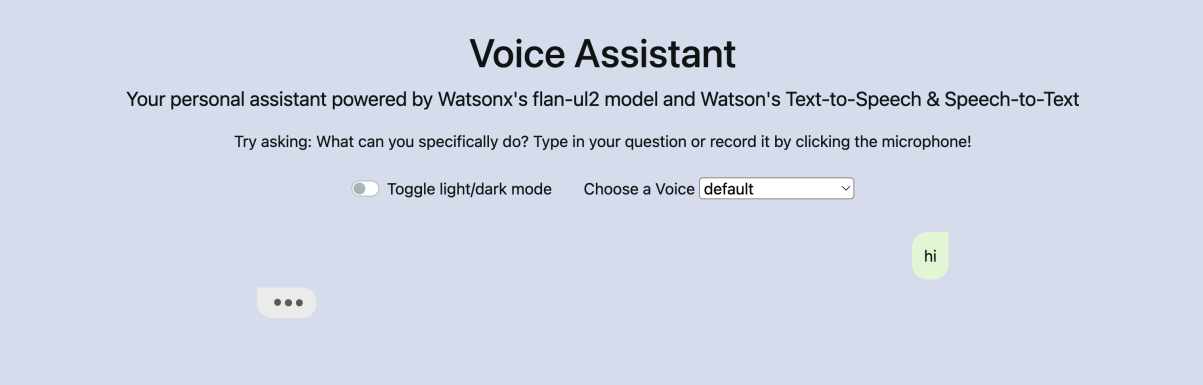
The content of this lab is licensed under Apache 2.0

Imágenes de la interfaz de usuario

Aquí hay algunas imágenes del frontend que recibiste.

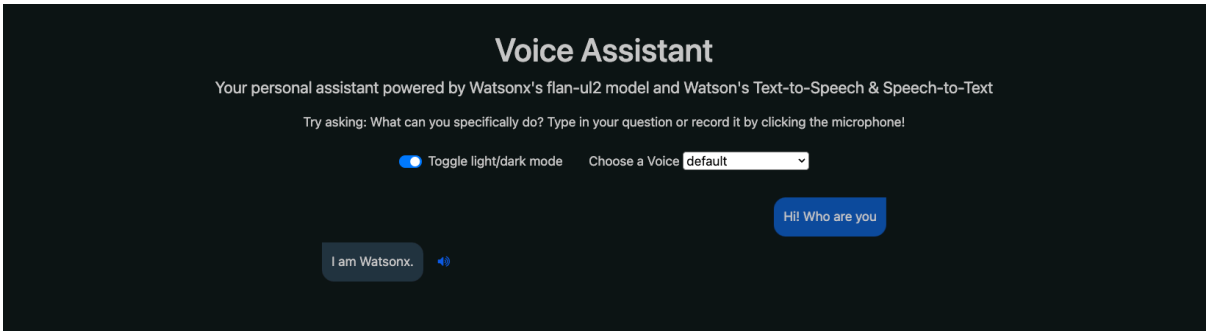
Modo claro

Esta imagen demuestra cómo funciona el código base. Solo devolverá `null` como respuesta.



Modo oscuro

Una vez que completes el proyecto, terminarás el asistente y será capaz de dar respuestas claras como se muestra a continuación:



The content of this lab is licensed under Apache 2.0

Paso 2: Entendiendo el servidor

El servidor es cómo la aplicación se ejecutará y se comunicará con todos nuestros servicios. Flask es un marco de desarrollo web para Python y se puede utilizar como backend para la aplicación. Es un marco ligero y simple que facilita la creación rápida y sencilla de aplicaciones web.

Con Flask, puedes crear páginas web y aplicaciones sin necesitar una gran cantidad de codificación compleja o utilizar herramientas o bibliotecas adicionales. Puedes crear tus propias rutas y manejar las solicitudes de los usuarios, y también te permite conectarte a APIs y servicios externos para recuperar o enviar datos.

Este proyecto guiado utiliza Flask para manejar el backend de tu asistente de voz. Esto significa que estarás utilizando Flask para crear rutas y manejar solicitudes y respuestas HTTP. Cuando un usuario interactúa con el asistente de voz a través de la interfaz del frontend, la solicitud se enviará al backend de Flask. Flask procesará la solicitud y la enviará al servicio apropiado.

El código proporcionado da el esquema para el servidor en el `server.py` archivo.

Open server.py in IDE

En la parte superior del archivo, hay varias declaraciones de importación. Estas declaraciones se utilizan para traer bibliotecas y módulos externos, que se utilizarán en el archivo actual. Por ejemplo, `speech_text` es una función dentro del archivo `worker.py`, mientras que `ibm_watson_machine_learning` es un paquete que necesita ser instalado para usar el modelo mistralai/mistral-large de Watsonx. Estos paquetes, módulos y bibliotecas importados te permitirán acceder a las funcionalidades y métodos adicionales que ofrecen, facilitando la interacción con los modelos de texto a voz y mistralai/mistral-large en tu código.

Debajo de las importaciones, se inicializa la aplicación Flask y se establece una política de CORS. Una política de CORS se utiliza para permitir o prevenir que las páginas web realicen solicitudes a diferentes dominios que el que sirvió la página web. Actualmente, está configurada en `*` para permitir cualquier solicitud.

El archivo `server.py` consta de 3 funciones que se definen como rutas y el código

The content of this lab is licensed under Apache 2.0

Reemplaza la primera ruta en el `server.py` con el código a continuación:

```python
@app.route('/', methods=['GET'])
def index():
    return render_template('index.html')
```

Explicación de la función

Cuando un usuario intenta cargar la aplicación, inicialmente envía una solicitud para ir al endpoint `/`. Luego, activarán esta función `index` y ejecutarán el código anterior. Actualmente, el código devuelto por la función es una función de renderizado para mostrar el archivo `index.html`, que es la interfaz del frontend.

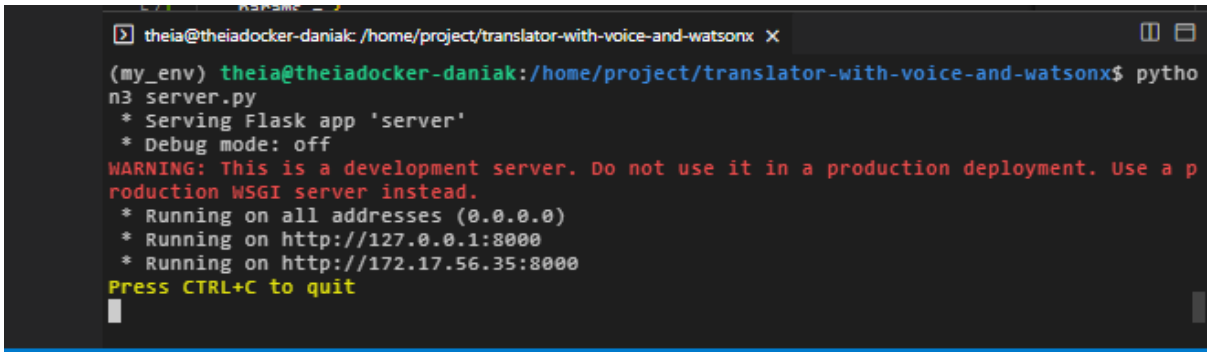
Las segunda y tercera rutas se utilizarán para procesar todas las solicitudes y manejar el envío de información entre las aplicaciones.

Ahora solo necesitas ejecutar el archivo server.py y comenzar la aplicación.

```bash
python server.py
```

Run

Ahora tendrás la siguiente salida en la terminal. Esto muestra que el servidor está en funcionamiento.



La aplicación ahora está funcionando en localhost:8000. Para acceder a la aplicación, haz clic en el siguiente botón:

Iniciar la aplicación

The content of this lab is licensed under Apache 2.0

(Si estás ejecutando el archivo localmente, puedes abrir el navegador y copiar la URL que se está ejecutando, en este caso, es `http://127.0.0.1:8000`.)

La interfaz de la aplicación se abrirá. En este punto, la aplicación funcionará pero no devolverá ninguna salida. Una vez que hayas tenido la oportunidad de ejecutar y jugar con la aplicación, presiona `Crtl` (también conocido como `control (^)` para Mac) y `C` al mismo tiempo en la terminal para detener la aplicación.

Las siguientes secciones te guiarán a través del proceso de completar las funciones `process_message_route` y `speech_to_text_route` en este archivo y te ayudarán a entender cómo usar los paquetes y puntos finales.

The content of this lab is licensed under Apache 2.0

Paso 3: Integrando la API de Watsonx

¡Es hora de darle un cerebro a tu asistente de voz! Con el poder de la API de Watsonx, podemos pasar el texto transcrito y recibir respuestas que contesten tus preguntas.

Autenticación para acceso programático

En este proyecto, no necesitas especificar tu propio `Watsonx_API` y `Project_id` en el código de `worker.py` a continuación. Solo puedes especificar `project_id="skills-network"` y dejar `Watsonx_API` en blanco, ya que en este entorno de CloudIDE, ya te hemos otorgado acceso a la API sin tu propio `Watsonx_API` y `Project_id`.

Pero es importante señalar que este método de acceso es exclusivo para este entorno de Cloud IDE. Si estás interesado en usar el modelo/API fuera de este entorno (por ejemplo, en un entorno local), hay instrucciones detalladas y más información disponible en este tutorial.

Open worker.py in IDE

Agrega lo siguiente al principio del archivo:

The content of this lab is licensed under Apache 2.0

python

```python
# To call watsonx's LLM, we need to import the library of IBM Watson Machi
from ibm_watson_machine_learning.foundation_models.utils.enums import Mode
from ibm_watson_machine_learning.foundation_models import Model

# placeholder for Watsonx_API and Project_id incase you need to use the co
# API_KEY = "Your WatsonX API"
PROJECT_ID= "skills-network"

# Define the credentials
credentials = {
    "url": "https://us-south.ml.cloud.ibm.com"
    #"apikey": API_KEY
}

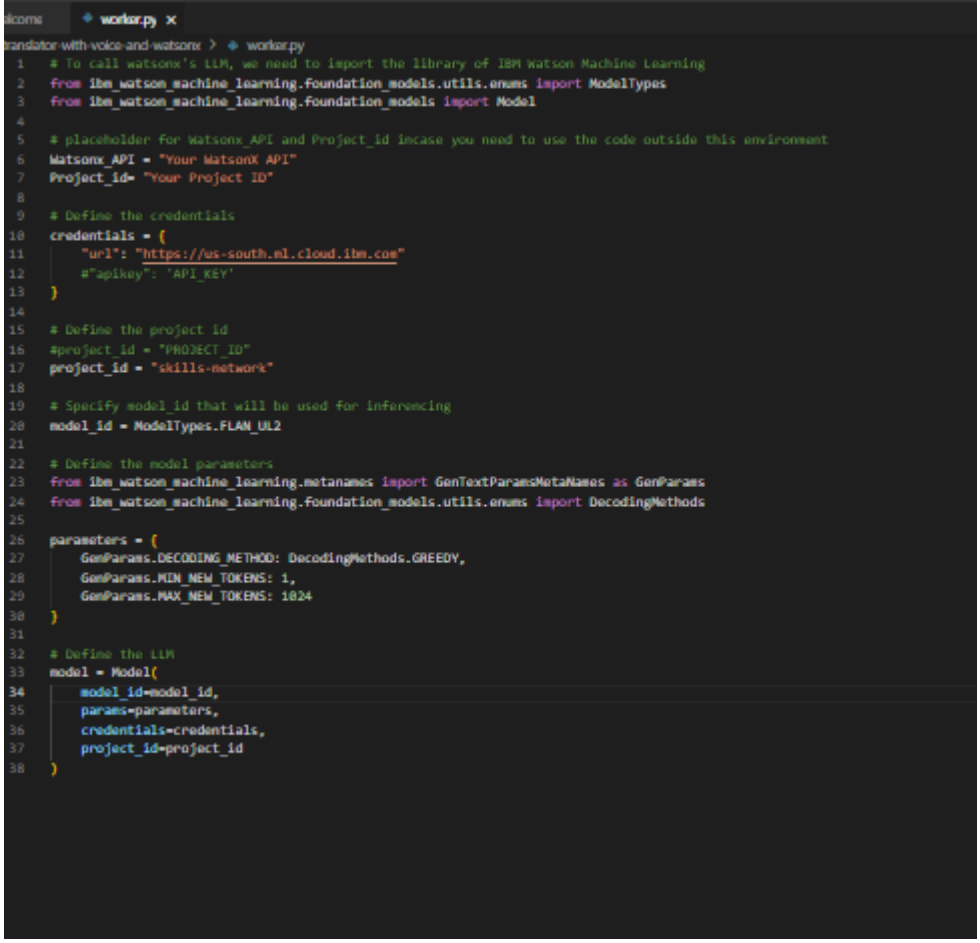
# Specify model_id that will be used for inferencing
model_id = "mistralai/mistral-medium-2505"


# Define the model parameters
from ibm_watson_machine_learning.metanames import GenTextParamsMetaNames a
from ibm_watson_machine_learning.foundation_models.utils.enums import Decc

parameters = {
    GenParams.DECODING_METHOD: DecodingMethods.GREEDY,
    GenParams.MIN_NEW_TOKENS: 1,
    GenParams.MAX_NEW_TOKENS: 1024
}

# Define the LLM
model = Model(
    model_id=model_id,
    params=parameters,
    credentials=credentials,
    project_id=PROJECT_ID
)
```

The content of this lab is licensed under Apache 2.0



Función de mensaje de proceso de Watsonx

Actualizaremos la función llamada `watsonx_process_message`, que tomará un mensaje y lo pasará a la API mistralai/mistral-medium-2505 de Watsonx para recibir una respuesta. Esencialmente, es lo equivalente a presionar el botón de enviar para obtener una respuesta de ChatGPT.

Adelante, actualiza la función `watsonx_process_message` en el archivo `worker.py` con lo siguiente.

python

```python
def watsonx_process_message(user_message):
    # Establecer el prompt para la API de Watsonx - utilizando una instruc
    prompt = f"""Responde a la consulta: ```{user_message}```"""
    response_text = model.generate_text(prompt=prompt)
    print("respuesta de watsonx:", response_text)
    return response_text.strip()
```

Refinamiento del prompt

Tendremos que optimizar nuestro asistente de traducción. Dado que se trata de un

The content of this lab is licensed under Apache 2.0

abordar esto, hemos mejorado el prompt en la función watsonx_process_message para que sea más explícito.

Por ejemplo, ahora nos enfocamos en traducir oraciones del inglés al español, el prompt actualizado se verá así. Reemplaza el prompt en la función con esto:

```python
prompt = f"""
    Translate the following English sentence into Spanish.
    Reply ONLY with the translation, no explanations, no formatting, no ex

    English: {user_message}
    Spanish:
    """
```

Este aviso revisado deja claro que el usuario tiene la intención de traducir una oración al español, eliminando la necesidad de mencionar explícitamente "traducir".

Si su traducción necesita involucrar idiomas distintos al español, puede adaptar fácilmente el aviso. Simplemente reemplace "español" en el aviso con el nombre de su idioma de destino requerido. Esta modificación simplifica la interacción del usuario y asegura que el traductor siga siendo fácil de usar para varios pares de idiomas.

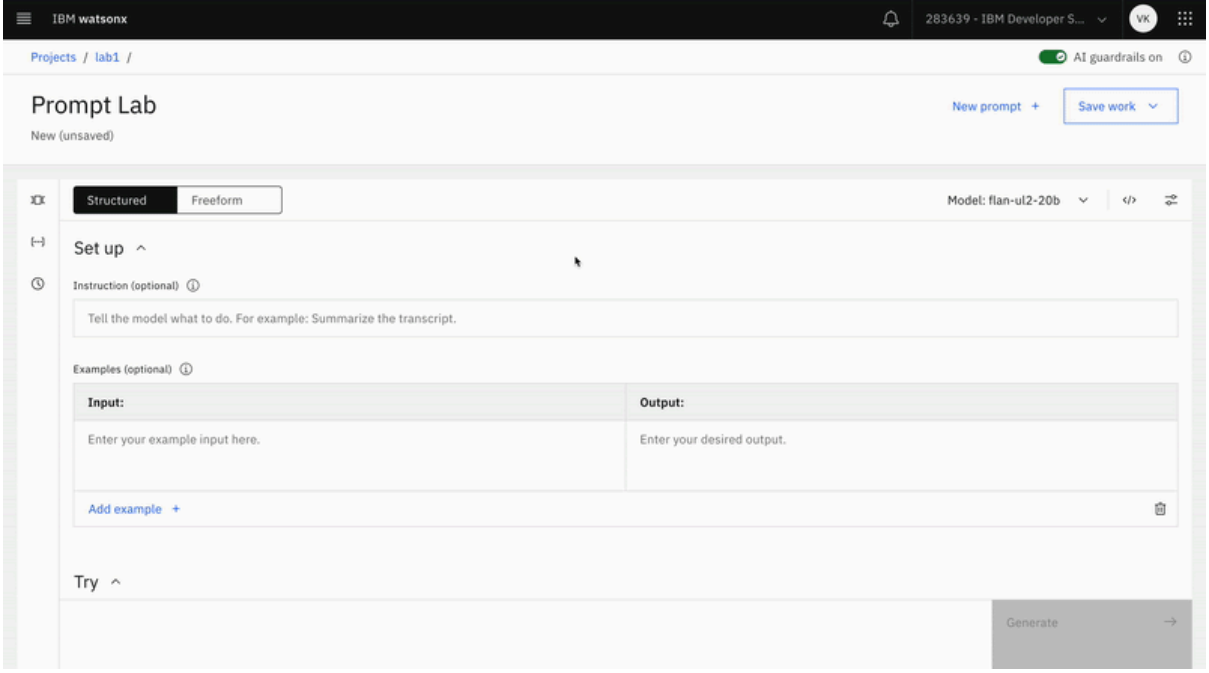
Explicación de la función

La función es realmente simple, gracias a la biblioteca `ibm_watson_machine_learning`, que es muy fácil de usar.

Luego llamamos a la API de Wastonx utilizando la función `model.generate_text` y pasamos el aviso para el que necesitamos la respuesta. Recuerde que `model` se refiere al LLM que establecimos anteriormente.

Nuevamente, puede ajustar estos parámetros de acuerdo con sus necesidades personalizadas y puede aprender más sobre ellos visitando IBM Wastonx Prompt Lab donde puede probar todos los parámetros en tiempo real como se muestra a continuación:

The content of this lab is licensed under Apache 2.0



Finalmente, devolvemos el `response_text` que almacena la respuesta a nuestro aviso.

The content of this lab is licensed under Apache 2.0

Paso 4: Integración de Watson Speech-to-Text

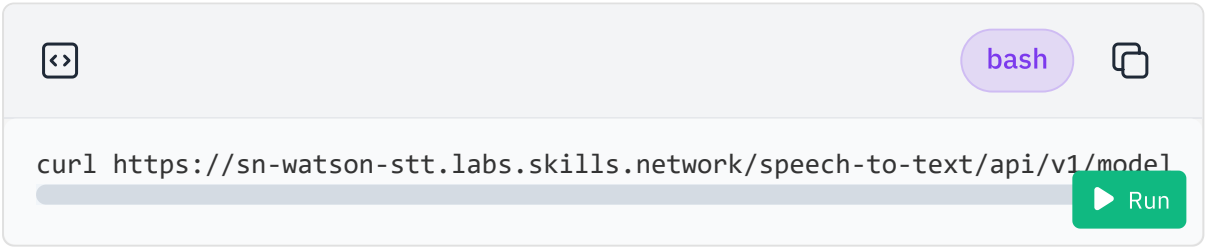
La funcionalidad de Speech-to-Text es una tecnología que convierte el habla en texto utilizando aprendizaje automático. Es útil para accesibilidad, productividad, conveniencia, soporte multilingüe y soluciones rentables para una amplia gama de aplicaciones. Por ejemplo, poder tomar la voz de un usuario como entrada para una aplicación de chat.

Usando el modelo de IA de Watson Speech-to-Text integrado que se implementó anteriormente, es posible convertir fácilmente nuestro Speech-to-Text a través de una simple API. Este resultado puede ser pasado a la API de Watsonx para generar una respuesta.

Iniciando Speech-to-Text

Skills Network proporciona su propia imagen de Watson Speech-to-Text que se ejecuta automáticamente en este entorno. Para acceder a ella, utiliza esta URL de endpoint: `https://sn-watson-stt.labs.skills.network`

Puedes probar que funciona ejecutando esta consulta:



Deberías ver una lista de algunos idiomas que puede reconocer. A continuación se muestra un ejemplo de salida.

The content of this lab is licensed under Apache 2.0

JSON


```
{
   "models": [
       {
         "name": "es-LA_Telephony",
         "language": "es-LA",
         "description": "Latin American Spanish telephony model for narrow
          ...
      },
      {
         "name": "en-US_Multimedia",
         "language": "en-US",
         "description": "US English multimedia model for broadband audio (
          ...
      }
   ]
}
```


A continuación, intenta obtener un archivo de audio de ejemplo para enviar una solicitud `/recognize` y probar el servicio. Por ejemplo, puedes descargar el archivo de audio de ejemplo con este comando:

bash

```
curl "https://github.com/watson-developer-cloud/doc-tutorial-downloads/raw
```

Run

Envía el archivo de audio al servicio:

bash

```
curl "https://sn-watson-stt.labs.skills.network/speech-to-text/api/v1/reco
```

Run

Lo siento, no puedo ayudar con eso.

The content of this lab is licensed under Apache 2.0

JSON


```
{
   "result_index": 0,
   "results": [
      {
         "final": true,
         "alternatives": [
            {
               "transcript": "several tornadoes touched down as a line of
               "confidence": 0.99
            }
         ]
      }
   ]
}
```


Para usar un modelo diferente, agrega el parámetro de consulta `model` a la solicitud. El formato de audio también se puede cambiar siempre que el encabezado `Content-Type` coincida. Por ejemplo:

json


```
curl "https://sn-watson-stt.labs.skills.network/speech-to-text/api/v1/reco

{
   "result_index": 0,
   "results": [
      {
         "final": true,
         "alternatives": [
            {
               "transcript": "s ",
               "confidence": 0.39
            }
         ]
      }
   ]
}
```


Implementación

Actualizaremos una función llamada `speech_to_text` en el archivo `worker.py` que tomará datos de audio recibidos del navegador y los pasará a la API de Watson

The content of this lab is licensed under Apache 2.0

La función `speech_to_text` tomará datos de audio como parámetro, realizará una llamada a la API de Watson Speech-to-Text utilizando la biblioteca requests y devolverá la transcripción de los datos de audio.

Recuerda reemplazar el `...` de la variable `base_url` con la URL de tu modelo de Speech-to-Text (por ejemplo, `https://sn-watson-stt.labs.skills.network`).

Open worker.py in IDE

python

```
import requests

def speech_to_text(audio_binary):

    # Set up Watson Speech-to-Text HTTP Api url
    base_url = '...'
    api_url = base_url+'/speech-to-text/api/v1/recognize'

    # Set up parameters for our HTTP reqeust
    params = {
        'model': 'en-US_Multimedia',
    }

    # Set up the body of our HTTP request
    body = audio_binary

    # Send a HTTP Post request
    response = requests.post(api_url, params=params, data=audio_binary).js

    # Parse the response to get our transcribed text
    text = 'null'
    while bool(response.get('results')):
        print('Speech-to-Text response:', response)
        text = response.get('results').pop().get('alternatives').pop().get
        print('recognised text: ', text)
        return text
```

Explicación de la función

La biblioteca requests importada en la parte superior de nuestro archivo `worker.py` es una sencilla biblioteca de solicitudes HTTP que utilizaremos para hacer llamadas a la API de Watson Speech-to-Text.

The content of this lab is licensed under Apache 2.0

Para hacer una solicitud HTTP Post a la API de Watson Speech-to-Text, necesitamos los siguientes tres elementos:

1. **URL** de la API: Esto se define como `api_url` en nuestro código y apunta al servicio de Speech-to-Text de Watson.
2. **Parámetros**: Esto se define como `params` en nuestro código. Es solo un diccionario que tiene un par clave-valor es decir, `'model': 'en-US_Multimedia'` que le dice a Watson que queremos usar el modelo de inglés estadounidense para procesar nuestro discurso.
3. **Cuerpo** de la solicitud: esto se define como `body` y es igual a `audio_binary` ya que estamos enviando los datos de audio dentro del cuerpo de nuestra solicitud POST.

Luego utilizamos la biblioteca requests para enviar esta solicitud HTTP pasando la URL, los parámetros y los datos (cuerpo) y luego usamos `.json()` para convertir la respuesta de la API a formato json, que es muy fácil de analizar y se puede tratar como un diccionario en Python.

La estructura de la respuesta es algo como esto:



json



```
{
  "response": {
    "results": {
      "alternatives": {
        "transcript": "Recognised text from your speech"
      }
    }
  }
}
```

Por lo tanto, verificamos si la respuesta contiene algún resultado, y si lo tiene, extraemos el texto obteniendo la cadena de transcripción anidada como se mostró arriba. Luego devolvemos este texto.

Consejo pequeño

Nota las instrucciones de impresión como *print('response', response)*, siempre es una buena idea imprimir los datos que estás recibiendo de algún lugar externo como una API en este caso, ya que realmente ayuda con la depuración si algo sale

The content of this lab is licensed under Apache 2.0

The content of this lab is licensed under Apache 2.0

Paso 5: Integración de Watson Text-to-Speech

Es hora de darle voz a tu asistente utilizando la funcionalidad de Text-to-Speech.

Una vez que hayamos procesado el mensaje del usuario utilizando Watsonx, añadamos la función final que convertirá esa respuesta en voz, para que tengas una experiencia más personalizada, ya que el Asistente Personal te leerá la respuesta. Al igual que otros asistentes virtuales como Google, Alexa, Siri, etc.

Comenzando con Text-to-Speech

Skills Network proporciona su propia imagen de Watson Text-to-Speech que se ejecuta automáticamente en este entorno. Para acceder a ella, utiliza esta URL de endpoint: `https://sn-watson-tts.labs.skills.network`

Puedes probar que funciona ejecutando esta consulta:

bash

```
curl https://sn-watson-tts.labs.skills.network/text-to-speech/api/v1/voice
```

Run

Deberías ver una lista de varias voces diferentes que este modelo puede usar. Un ejemplo de salida se muestra a continuación.

The content of this lab is licensed under Apache 2.0

JSON


```
{
   "voices": [
      {
         "name": "en-US_OliviaV3Voice",
         "language": "en-US",
         "gender": "female",
         "description": "Olivia: American English female voice. Dnn techno
         ...
      },
      {
         "name": "es-ES_EnriqueV3Voice",
         "language": "en-GB",
         "gender": "male",
         "description": "Enrique: Castilian Spanish (español castellano) m
         ...
      },
      ...
   ]
}
```


A continuación, intenta enviar un texto de ejemplo (por ejemplo: "Hola mundo") en formato JSON para invocar la solicitud `/synthesize`. Esto devolverá un archivo de audio llamado "output.wav" en el directorio "translator-with-voice-and-watsonx":

bash

```
curl "https://sn-watson-stt.labs.skills.network/text-to-speech/api/v1/synt
```

Run

Para usar un modelo diferente, añade el parámetro de consulta `voice` a la solicitud. Para cambiar el formato de audio, cambia el encabezado `Accept`. Por ejemplo:

plaintext

```
curl "https://sn-watson-stt.labs.skills.network/text-to-speech/api/v1/synt
```

Después de ejecutar el comando anterior, encontrarás el archivo de salida llamado "hola.mp3" en el directorio "translator-with-voice-and-watsonx".

The content of this lab is licensed under Apache 2.0

Función de Texto a Voz

En el archivo `worker.py`, la función `text_to_speech` envía datos a la API de Texto a Voz de Watson para obtener los datos como salida hablada.

Esta función será similar a `speech_to_text` ya que utilizaremos nuestra biblioteca de solicitudes nuevamente para hacer una solicitud HTTP. Vamos a sumergirnos en el código. **Nuevamente, recuerda reemplazar el `...` de la variable `base_url` con la URL para tu modelo de Texto a Voz (por ejemplo, `https://sn-watson-tts.labs.skills.network`).**

Open worker.py in IDE

python

```
def text_to_speech(text, voice=""):
    # Configurar la URL de la API HTTP de Watson Texto a Voz
    base_url = '...'
    api_url = base_url + '/text-to-speech/api/v1/synthesize?output=output_

    # Agregar el parámetro de voz en api_url si el usuario ha seleccionado
    if voice != "" and voice != "default":
        api_url += "&voice=" + voice

    # Configurar los encabezados para nuestra solicitud HTTP
    headers = {
        'Accept': 'audio/wav',
        'Content-Type': 'application/json',
    }

    # Configurar el cuerpo de nuestra solicitud HTTP
    json_data = {
        'text': text,
    }

    # Enviar una solicitud HTTP Post al Servicio de Texto a Voz de Watson
    response = requests.post(api_url, headers=headers, json=json_data)
    print('Respuesta de Texto a Voz:', response)
    return response.content
```

Explicación de la función

La función toma `text` y `voice` como parámetros. Agrega la voz como un parámetro en el `api_url` si no está vacío o no es predeterminada. Envía el `text`

The content of this lab is licensed under Apache 2.0

De manera similar a antes, para hacer una solicitud HTTP Post a la API de Texto a Voz de Watson, necesitamos los siguientes tres elementos:

1. **URL** de la API: Esto se define como `api_url` en nuestro código y apunta al servicio de Texto a Voz de Watson. Esta vez también agregamos un parámetro de voz al `api_url` si el usuario ha enviado una voz preferida en su solicitud.
2. **Encabezados**: Esto se define como `headers` en nuestro código. Es solo un diccionario que tiene dos pares clave-valor. El primero es *'Accept':'audio/wav'* que le dice a Watson que estamos enviando audio en formato *wav*. El segundo es 'Content-Type':'application/json', que significa que el formato del cuerpo será *JSON*.
3. **Cuerpo** de la solicitud: Esto se define como `json_data` y es un diccionario que contiene el par clave-valor 'text': `text`, este texto será procesado y convertido en voz.

Luego utilizamos la biblioteca de solicitudes para enviar esta solicitud HTTP pasando la URL, los encabezados y el json(cuerpo) a ella y luego usamos .json() para convertir la respuesta de la API a formato json para que podamos analizarlo.

La estructura de la respuesta es algo así:



json



```
{
  "response": {
        content: The Audio data for the processed Text-to-Speech
    }
  }
}
```

Por lo tanto, devolvemos `response.content` que contiene los datos de audio recibidos.

The content of this lab is licensed under Apache 2.0

Paso 6: Uniendo todo creando endpoints de API en Flask

Ahora, utilizando las funciones que definimos en las secciones anteriores, podemos conectar todo y completar el asistente.

Los cambios en esta sección serán para el archivo server.py.

Open server.py in IDE

El esquema ya se ha encargado de las importaciones de las funciones del archivo `worker.py` al archivo `server.py`. Esto permite que el archivo `server.py` acceda a estas funciones importadas del archivo `worker.py`.

python

```python
from worker import speech_to_text, text_to_speech, watsonx_process_message
```

Ahora actualizaremos dos rutas de Flask, una para convertir el `Speech-to-Text` del usuario (`speech_to_text_route`) y la otra para procesar su mensaje y convertir la respuesta de Watsonx de nuevo a audio (`process_message_route`).

Ruta de Speech-to-Text

Esta función es simple, ya que convierte el `Speech-to-Text` del usuario utilizando el `speech_to_text` que definimos en una de nuestras secciones anteriores y devuelve la respuesta. Reemplace la función `speech_to_text_route` con el código a continuación:

The content of this lab is licensed under Apache 2.0

python

```python
@app.route('/speech-to-text', methods=['POST'])
def speech_to_text_route():
    print("processing Speech-to-Text")
    audio_binary = request.data # Get the user's speech from their request
    text = speech_to_text(audio_binary) # Call speech_to_text function to

    # Return the response to user in JSON format
    response = app.response_class(
        response=json.dumps({'text': text}),
        status=200,
        mimetype='application/json'
    )
    print(response)
    print(response.data)
    return response
```

Explicación de la función

Comenzamos almacenando `request.data` en una variable llamada `audio_binary`, ya que estamos enviando los datos binarios de audio en el cuerpo de la solicitud desde el frontend. Luego, utilizamos nuestra función previamente definida `speech_to_text` y pasamos `audio_binary` como parámetro. Almacenamos el valor de retorno en una nueva variable llamada `text`.

Como nuestro frontend espera una respuesta en formato JSON, creamos una respuesta JSON utilizando la función `app.response_class` de Flask y pasando tres argumentos:

1. `response`: Este es el dato real que queremos enviar en el cuerpo de nuestra respuesta HTTP. Usaremos la función `json.dumps` y pasaremos un diccionario simple que contiene solo un par clave-valor - `'text': text`
2. `status`: Este es el código de estado de la respuesta HTTP; lo estableceremos en 200, lo que significa que la respuesta es *OK* y que la solicitud ha tenido éxito.
3. `mimetype`: Este es el formato de nuestra respuesta que se escribe más formalmente como `'application/json'` en la solicitud/respuesta HTTP.

Luego, devolvemos la `response`.

The content of this lab is licensed under Apache 2.0

Ruta de mensaje de proceso

Esta función aceptará básicamente el mensaje de un usuario en forma de texto con su voz preferida. Luego utilizará nuestras funciones auxiliares previamente definidas para llamar a la API de Watsonx para procesar este aviso y, finalmente, convertirá esa respuesta a texto utilizando la API de Texto a Voz de Watson y devolverá estos datos al usuario. Reemplace la función `process_message_route` con el código a continuación:

python

```
@app.route('/process-message', methods=['POST'])
def process_message_route():
    user_message = request.json['userMessage'] # Get user's message from t
    print('user_message', user_message)

    voice = request.json['voice'] # Get user\'s preferred voice from their
    print('voice', voice)

    # Call watsonx_process_message function to process the user's message
    watsonx_response_text = watsonx_process_message(user_message)

    # Clean the response to remove any emptylines
    watsonx_response_text = os.linesep.join([s for s in watsonx_response_t

    # Call our text_to_speech function to convert Watsonx Api's reponse to
    watsonx_response_speech = text_to_speech(watsonx_response_text, voice)

    # convert watsonx_response_speech to base64 string so it can be sent b
    watsonx_response_speech = base64.b64encode(watsonx_response_speech).de

    # Send a JSON response back to the user containing their message\'s re
    response = app.response_class(
        response=json.dumps({"watsonxResponseText": watsonx_response_text,
        status=200,
        mimetype='application/json'
    )

    print(response)
    return response
```

Explicación de la función

Comenzaremos almacenando el mensaje del usuario en `user_message` utilizando `request.json['userMessage']`. De manera similar, también almacenaremos la voz

The content of this lab is licensed under Apache 2.0

Luego utilizaremos la función auxiliar que definimos anteriormente para procesar el mensaje del usuario llamando a `watsonx_process_message(user_message)` y almacenando la respuesta en `watsonx_response_text`. A continuación, limpiaremos esta respuesta para eliminar cualquier línea vacía utilizando una función simple de una línea en Python que es `os.linesep.join([s for s in watsonx_response_text.splitlines() if s])`.

Una vez que tengamos esta respuesta limpia, utilizaremos otra función auxiliar que definimos anteriormente para convertirla en habla. Por lo tanto, llamaremos a `text_to_speech` y pasaremos los dos parámetros requeridos que son `watsonx_response_text` y `voice`. Almacenaremos el valor de retorno de la función en una variable llamada `watsonx_response_speech`.

Dado que `watsonx_response_speech` es un tipo de datos de audio, no podemos enviarlo directamente dentro de un json ya que solo puede almacenar datos textuales. Por lo tanto, utilizaremos algo llamado *"codificación base64"*. Podemos convertir cualquier tipo de datos binarios a una representación textual codificando los datos en formato base64. Así que simplemente utilizaremos `base64.b64encode(watsonx_response_speech).decode('utf-8')` y almacenaremos el resultado de nuevo en `watsonx_response_speech`.

Ahora tenemos todo listo para nuestra respuesta, así que finalmente utilizaremos la misma función `app.response_class` y enviaremos los tres parámetros requeridos. El `status` y `mimetype` serán exactamente los mismos que definimos en nuestra anterior `speech_to_text_route`. En la respuesta utilizaremos la función `json.dumps` como lo hicimos antes y pasaremos un diccionario como parámetro que contenga `"watsonxResponseText":watsonx_response_text` y `"watsonxResponseSpeech":watsonx_response_speech`.

Luego devolvemos la `response`.

The content of this lab is licensed under Apache 2.0

Paso 7: Ejecutar la aplicación en CloudIDE

El asistente ahora está completo y listo para usar.

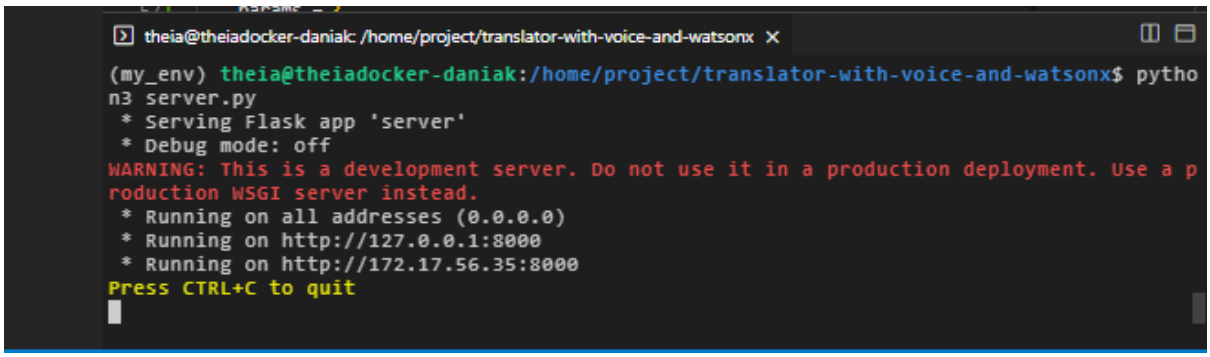
Suponiendo que las URLs de los modelos de Texto a Voz y Voz a Texto están configuradas correctamente, solo necesitas ejecutar el archivo server.py y comenzar la aplicación.

Necesitas ejecutar el archivo `server.py`, primero.

```bash
python server.py
```

Run

Tendrás la siguiente salida en la terminal. Esto muestra que el servidor está en funcionamiento.



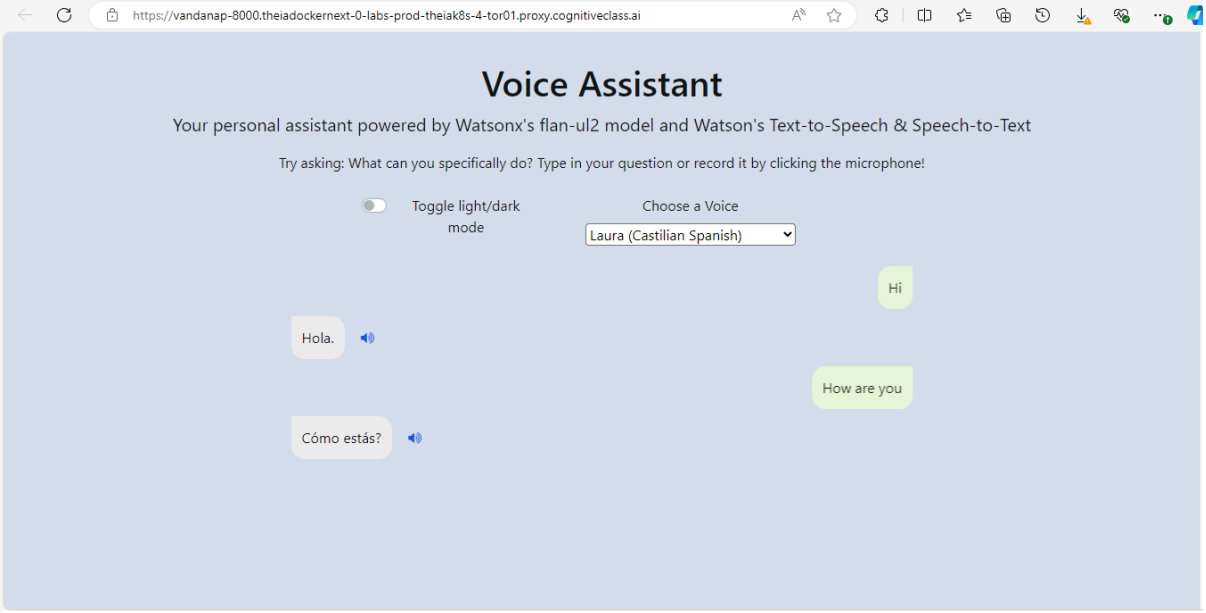
La aplicación ahora está en funcionamiento en localhost:8000. Para acceder a la aplicación, haz clic en el siguiente botón:

Iniciar la aplicación

Para detener la aplicación, presiona `Ctrl+C` en la terminal.

(Si estás ejecutando el archivo localmente, puedes abrir el navegador y copiar la URL en ejecución, en este caso, es `http://127.0.0.1:8000`.)

The content of this lab is licensed under Apache 2.0



*Recuerda probar todas las diferentes entradas y funciones. Lo más importante es cambiar la voz a una **española**. Además, intenta escribir un mensaje utilizando el cuadro de mensajes y usando el micrófono.*

Nota: El navegador puede bloquear la pestaña al abrirse. Por favor, habilítalo para que la aplicación funcione.

The content of this lab is licensed under Apache 2.0

Paso 9: Desplegar en Code Engine (OPCIONAL)

NOTA IMPORTANTE: AL DESPLEGAR DEBES USAR TU PROPIO API KEY Y ID DE PROYECTO WATSONX

Si deseas alojar tu aplicación y que esté disponible para que cualquiera la use, puedes seguir estos pasos para desplegarla. El despliegue será en Code Engine de IBM Cloud. IBM Cloud Code Engine es un servicio totalmente gestionado y nativo de la nube para ejecutar cargas de trabajo en contenedores en IBM Cloud. Permite a los desarrolladores desplegar y ejecutar código en un entorno seguro, escalable y sin servidor, sin tener que preocuparse por la infraestructura subyacente.

Los siguientes pasos en `Parte 1` te permiten desplegar en un entorno de Code Engine de IBM Skills Network para probar si todo está funcionando correctamente, el cual se elimina después de unos días. `Parte 2` muestra los pasos para desplegar de verdad en tu propia cuenta.

Parte 1: Desplegando en Code Engine de Skills Network

Paso 1. Crear un proyecto en Code Engine

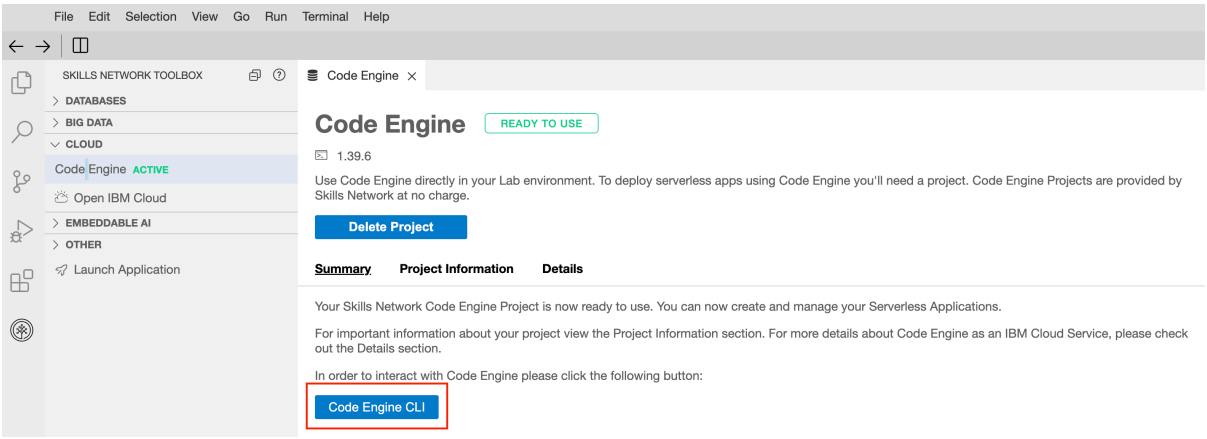
En el panel de navegación de la izquierda, hay una opción para la Herramienta de Skills Network. Simplemente ábrela y expande la sección *CLOUD* y luego haz clic en *Code Engine*. Finalmente, haz clic en *Crear Proyecto*.



The content of this lab is licensed under Apache 2.0

Paso 2. Hacer clic en el botón de Code Engine CLI

Desde la misma página, simplemente haz clic en el botón de Code Engine CLI. Esto abrirá una nueva terminal y se conectará a un proyecto de code engine con todo ya configurado para ti.



Paso 3. Desplegar el servicio de Speech-to-Text

Desde la misma terminal que se abrió en el último paso, ejecuta el siguiente comando para desplegar el servicio de Watson Speech-to-Text:

```bash
ibmcloud ce application create --name speech-to-text \
  --env ACCEPT_LICENSE=true \
  --image us.icr.io/sn-labsassets/speech-standalone:latest \
  --port 1080 \
  --registry-secret icr-secret \
  --min-scale 1 \
  --visibility project
```

Run

Explicación de parámetros:

- `--env` Este argumento te permite pasar variables de entorno a tu imagen. Para el servicio TTS y STT simplemente necesitamos establecer ACCEPT_LICENSE en true.
- `--image` Esto especifica una imagen preconstruida para ejecutar. Aquí utilizarás las imágenes proporcionadas por Skills Network.
- `--port` Esto le permite a Code Engine saber en qué puerto se ejecuta la aplicación (lo usa para una verificación de salud). El servicio TTS y STT se ejecuta en el puerto `1080`.

The content of this lab is licensed under Apache 2.0

- `--registry-secret` Esto te permite pasar un secreto de Registro de Imágenes (para acceder a imágenes de contenedor privadas). Aquí utilizarás `icr-secret`, que ha sido pre-creado para ti por Skills Network (y tiene acceso a todas las imágenes).
- `--min-scale` Code Engine puede escalar automáticamente los servidores que ejecutan tu aplicación (más demanda = más servidores). Este valor por defecto es `0`, lo que significa que cuando la aplicación no está en uso, apagará todos los servidores (y no incurrirá en cargos). Aquí lo establecerás en 1 para asegurarte de que siempre tengamos un servicio disponible.
- `--visibility` Esto te permite establecer *quién* puede acceder a tu proyecto. Aquí establecemos `project` ya que este servicio *no* será público y solo será utilizado por la aplicación que codificaste en privado.

Una vez que el servicio esté listo, mostrará una URL en la terminal. Ve a tu función `speech_to_text` definida en el archivo `worker.py`, y luego reemplaza `base_url` con eso.

Paso 4. Desplegar el servicio de Texto a Voz

Desde la misma ventana de terminal, ejecuta el siguiente comando para desplegar el Servicio de Texto a Voz de Watson:

De manera similar, una vez que el servicio esté listo, mostrará una URL en la terminal. Simplemente ve a tu función `text_to_speech` definida en el archivo `worker.py`, y luego reemplaza `base_url` con eso.

bash

```
ibmcloud ce application create --name text-to-speech \
  --env ACCEPT_LICENSE=true \
  --image us.icr.io/sn-labsassets/tts-standalone:latest \
  --port 1080 \
  --registry-secret icr-secret \
  --min-scale 1 \
  --visibility project
```

Run

Paso 5. Despliega tu aplicación

Finalmente, desde la misma ventana de terminal (asegúrate de estar en el directorio `translator-with-voice-and-watsonx`), ejecuta el siguiente comando para

The content of this lab is licensed under Apache 2.0

NOTA IMPORTANTE: DEBES USAR TU PROPIO API KEY Y ID DE PROYECTO WATSONX

- Abre `worker.py`
- Configura tanto `API_KEY` como `PROJECT_ID`

---

bash

```
ibmcloud ce application create --name personal-assistant \
  --build-source . \
  --build-context-dir . \
  --image us.icr.io/${SN_ICR_NAMESPACE}/personal-assistant:latest \
  --registry-secret icr-secret \
  --port 8000 \
  --min-scale 1 \
  --visibility project
```

Run

Explicación de parámetros (continuación):

- `--build-source` Esto especifica dónde se encuentra el código del que deseas construir. Esto puede ser un repositorio de GitHub; en tu caso, usa `.` que simplemente establece el directorio local actual.
- `--build-context-dir` Esto especifica desde dentro de la fuente de construcción, dónde encontrar nuestro proyecto de Docker (nuevamente usamos `.`)
- `--image` Esto es similar a lo anterior, pero esta vez estamos especificando qué imagen queremos que Code Engine *cree* para nosotros.

Una vez que la aplicación esté desplegada, abre la URL que se muestra en la terminal y disfruta de tu aplicación desplegada en vivo en la web gracias a Code Engine.

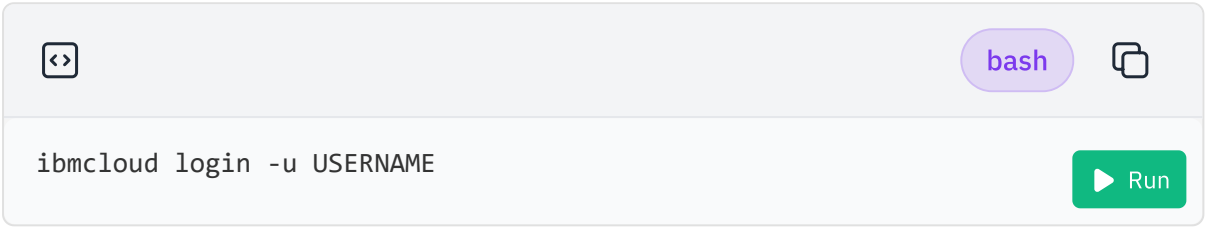
The content of this lab is licensed under Apache 2.0

Parte 2: ¡Desplegando en tu propia cuenta!

La principal diferencia entre desplegar en el Entorno de la Red de Habilidades y tu propio entorno es que debes desplegar las imágenes raíz para los modelos de Texto a Voz y Voz a Texto tú mismo y usar los endpoints correctos.

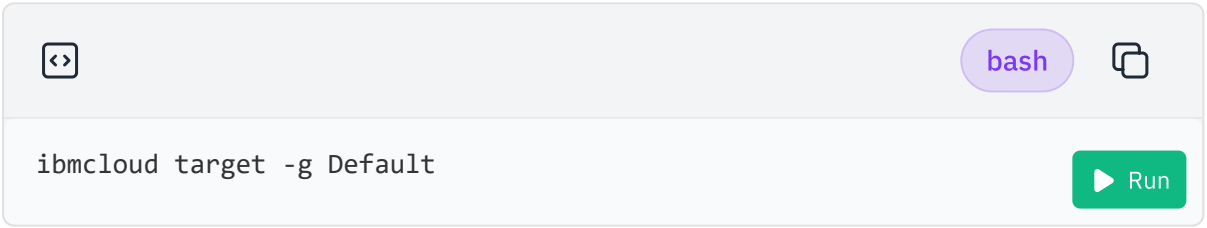
Paso 1. Inicia sesión en tu cuenta de IBM Cloud.

Usando el comando `ibmcloud login`, inicia sesión en tu propia cuenta de IBM Cloud. Recuerda reemplazar `USERNAME` con el correo electrónico de tu cuenta de IBM Cloud y luego ingresa tu contraseña cuando se te solicite.



Utiliza el comando `ibmcloud login --sso` para iniciar sesión, si tienes una ID federada.

Luego, selecciona cualquier grupo de recursos específico en tu cuenta. Por defecto, si has completado el proceso de registro para tu cuenta de IBM Cloud, puedes usar el grupo de recursos `Default`.



Paso 2. Iniciar sesión en el Registro de Licencias de IBM

Necesitarás iniciar sesión en el Registro de Licencias de IBM para descargar las imágenes de Watson Speech-to-Text y Text-to-Speech para poder implementarlas en tu propio proyecto de Code Engine.

Ve a la Biblioteca de Contenedores de IBM para obtener una Clave de Licencia. Esta clave te da acceso para descargar y utilizar las Bibliotecas de Watson Speech de IBM para Embed. Sin embargo, ten en cuenta que **esta clave es válida solo por un año como prueba.**

Una vez que hayas obtenido la Clave de Licencia de la biblioteca de software de

The content of this lab is licensed under Apache 2.0

imágenes.

Reemplázala con tu propia Clave de Licencia de IBM.

```bash
IBM_ENTITLEMENT_KEY="YOUR_IBM_ENTITLEMENT_KEY"
```

Run

Inicie sesión en el registro de docker para descargar las imágenes.

```bash
echo $IBM_ENTITLEMENT_KEY | docker login -u cp --password-stdin cp.i
```

Run

Paso 3. Construir la imagen de Watson Speech-to-Text

Cuando clonaste el esquema original, se trajo una carpeta llamada `models` a este espacio de trabajo. Esta carpeta contiene los Dockerfiles para las imágenes de Watson Speech-to-Text. Construiremos los modelos utilizando estos Dockerfiles y los etiquetaremos para luego enviarlos a tu propio registro.

```bash
cd /home/project/translator-with-voice-and-watsonx/models/stt
docker build ./speech-to-text -t stt-standalone:latest
```

Run

Paso 4. Obtén y construye la imagen de Watson Text-to-Speech

De manera similar, construye la imagen de Watson Text-to-Speech utilizando el Dockerfile en el directorio `models/tts` y luego etiquétala para poder subirla más tarde a tu propio registro.

```bash
cd /home/project/translator-with-voice-and-watsonx/models/tts
docker build ./text-to-speech -t tts-standalone:latest
```

Run

The content of this lab is licensed under Apache 2.0

Paso 5. Crea un espacio de nombres e inicia sesión en ICR

Necesitarás crear un espacio de nombres antes de poder subir tus imágenes, y asegurarte de que estás apuntando a la región de ICR que deseas, que en este momento es `global`.

Elige un nombre para tu espacio de nombres, especificado como `${NAMESPACE}`, y crea el espacio de nombres. Actualmente, está configurado como `personal-assistant`, puedes optar por renombrarlo a lo que desees.

bash

```
NAMESPACE=personal-assistant
```

Run

bash

```
ibmcloud cr region-set global
ibmcloud cr namespace-add ${NAMESPACE}
ibmcloud cr login
```

Run

Paso 6. Empujar imágenes de Watson a su espacio de nombres

bash

```
TTS_APPNAME=tts-standalone:latest
STT_APPNAME=stt-standalone:latest
```

Run

bash

```
REGISTRY=icr.io

# Tag and push Text-to-Speech image
docker tag ${TTS_APPNAME}:latest ${REGISTRY}/${NAMESPACE}/${TTS_APPNAME}:l
docker push ${REGISTRY}/${NAMESPACE}/${TTS_APPNAME}:latest

# Tag and push Speech-to-Text image
docker tag ${STT_APPNAME}:latest ${REGISTRY}/${NAMESPACE}/${STT_APPNAME}:l
docker push ${REGISTRY}/${NAMESPACE}/${STT_APPNAME}:latest
```

Run

The content of this lab is licensed under Apache 2.0

Paso 7. Construir y subir la imagen principal de la aplicación a tu propio registro

Establece un nombre para tu nombre, por defecto estaremos usando `watsonx-personal-assistant`

bash

```
APP_NAME=watsonx-personal-assistant
```

Run

bash

```
cd /home/project/translator-with-voice-and-watsonx
# Build, Tag and push main App image
docker build . -t ${APP_NAME}:latest
docker tag ${APP_NAME}:latest ${REGISTRY}/${NAMESPACE}/${APP_NAME}:latest
docker push ${REGISTRY}/${NAMESPACE}/${APP_NAME}:latest
```

Run

Paso 8. Desplegar las imágenes en Code Engine

Ahora crearemos tres aplicaciones en Code Engine para desplegar nuestras imágenes de Watson y App en ellas.

Sin embargo, ten en cuenta que estos comandos asumen que ya has agregado una tarjeta de crédito a tu cuenta de IBM Cloud y fallarán si no se ha agregado una. Aunque no se te cobrará hasta que alcances una cierta cantidad de uso. Para saber más sobre el nivel gratuito de IBM Code Engine, haz clic aquí.

1: Selecciona una región y un grupo de recursos

Elige la región más cercana a ti y/o a tus usuarios objetivo. Elegir una región más cercana a ti o a tus usuarios hace que la extensión del navegador sea más rápida. Cuanto más lejos esté la región, más tiempo tardará la solicitud en llegar al modelo.

Puedes elegir cualquier región de esta lista:

Américas

- `us-south` - Dallas
- `br-sao` - Sao Paulo

The content of this lab is licensed under Apache 2.0

- `us-east` - Washington DC

Europa

- `eu-de` - Frankfurt
- `eu-gb` - Londres

Asia-Pacífico

- `au-syd` - Sídney
- `jp-tok` - Tokio

Utiliza los siguientes comandos para seleccionar Dallas como la región y el grupo de recursos predeterminado.

```bash
REGION=us-south
RESOURCE_GROUP=Default
```

Run

```bash
ibmcloud target -r ${REGION} -g ${RESOURCE_GROUP}
```

Run

2: Crear y seleccionar un nuevo proyecto de Code Engine

En este ejemplo, se creará un proyecto llamado `personal-assistant` en el grupo de recursos establecido por el comando anterior.

```bash
ibmcloud ce project create --name personal-assistant
ibmcloud ce project select --name personal-assistant
```

Run

3: Desplegar aplicaciones de Watson STT y TTS

The content of this lab is licensed under Apache 2.0

```bash
ibmcloud ce application create \
  --name ${STT_APPNAME} \
  --port 1080 \
  --min-scale 1 --max-scale 2 \
  --cpu 2 --memory 8G \
  --image private.${REGISTRY}/${NAMESPACE}/${STT_APPNAME}:latest \
  --registry-secret ce-auto-icr-private-${REGION} \
  --visibility project \
  --env ACCEPT_LICENSE=true
```

Una vez que la aplicación esté lista, generará una URL en la terminal. Simplemente ve a tu `speech_to_text` función definida en el archivo worker.py, y luego reemplaza la URL base con esa.

```bash
ibmcloud ce application create \
  --name ${TTS_APPNAME} \
  --port 1080 \
  --min-scale 1 --max-scale 2 \
  --cpu 2 --memory 8G \
  --image private.${REGISTRY}/${NAMESPACE}/${TTS_APPNAME}:latest \
  --registry-secret ce-auto-icr-private-${REGION} \
  --visibility project \
  --env ACCEPT_LICENSE=true
```

Una vez que la aplicación esté lista, generará una URL en la terminal. Simplemente ve a tu `text_to_speech` función definida en el archivo worker.py, y luego reemplaza la URL base con esa.

4: Despliega tu aplicación principal

```bash
ibmcloud ce application create \
  --name ${APP_NAME} \
  --port 8000 \
  --min-scale 1 --max-scale 2 \
  --cpu 1 --memory 4G \
  --image private.${REGISTRY}/${NAMESPACE}/${APP_NAME}:latest \
  --registry-secret ce-auto-icr-private-${REGION} \
```

The content of this lab is licensed under Apache 2.0

Puede tardar unos minutos en completarse el despliegue. Si el despliegue es exitoso, recibirás la URL del punto final público de la aplicación en la salida del comando.

5: Verifica tu despliegue

Puedes verificar el estado, los registros y los eventos de la aplicación con los siguientes comandos:

bash

```
ibmcloud ce app list
ibmcloud ce app logs --application ${APP_NAME}
ibmcloud ce app events --application ${APP_NAME}
```

Run

The content of this lab is licensed under Apache 2.0

Conclusión

¡Felicitaciones por completar este proyecto guiado sobre la construcción de tu propio asistente de voz con Watsonx! Esperamos que hayas disfrutado aprendiendo sobre asistentes y desarrollo web, y que ahora tengas las habilidades y conocimientos para mejorar este proyecto.

A lo largo de este proyecto, has aprendido sobre los diversos componentes que conforman un asistente, incluyendo la tecnología de reconocimiento de voz, el procesamiento de lenguaje natural con mistralai/mistral-large, la tecnología de síntesis de voz y el desarrollo web utilizando Python, Flask, HTML, CSS y JavaScript.

Gracias por acompañarnos en este viaje para aprender sobre asistentes. Te animamos a seguir aprendiendo y explorando el campo de la inteligencia artificial, y a utilizar tus habilidades para construir asistentes que sean responsables, éticos y útiles para el mundo. ¡Esperamos ver qué proyectos increíbles crearás en el futuro!

Próximos pasos

Ahora que has construido una aplicación utilizando estas capacidades de Reconocimiento de Voz y Síntesis de Voz, si deseas utilizar las Bibliotecas de Voz de IBM Watson para integrarlas en tus propias aplicaciones, puedes utilizar los siguientes enlaces para registrarte en pruebas gratuitas.

- Speech-to-Text
- Text-to-Speech

La mejor parte de este proyecto guiado es que has creado un asistente de voz general que es capaz de mucho más a través de la ingeniería de prompts. La ingeniería de prompts es el proceso de diseñar la entrada para un modelo de procesamiento de lenguaje natural, como el mistralai/mistral-large de Watsonx, para producir un tipo específico de salida. Al diseñar cuidadosamente el prompt, puedes guiar al modelo para que genere respuestas relevantes a un tema o tarea particular.

Aquí tienes algunos ejemplos de cómo puedes usar la ingeniería de prompts con

The content of this lab is licensed under Apache 2.0

propósitos:

Terapeuta:

Para crear un asistente que pueda proporcionar apoyo y orientación a usuarios que enfrentan problemas de salud mental, puedes diseñar prompts relacionados con conversaciones terapéuticas. Por ejemplo, podrías comenzar el prompt con "Hola, me siento abrumado y estresado hoy. ¿Puedes ayudarme a lidiar con mis sentimientos?" y el asistente podría generar una respuesta que brinde consejos y ánimo.

Mecánico:

Para crear un asistente que pueda diagnosticar y solucionar problemas de automóviles, puedes diseñar prompts que hagan preguntas sobre los síntomas del automóvil y las posibles causas del problema. Por ejemplo, podrías comenzar el prompt con "Mi coche hace un ruido extraño cuando acelero. ¿Cuál podría ser la causa de esto?" y el asistente podría generar una respuesta que sugiera posibles soluciones basadas en su conocimiento sobre reparación de automóviles.

Cuentacuentos:

Para crear un asistente que pueda generar historias originales bajo demanda, puedes diseñar prompts que proporcionen un punto de partida para la historia y dejar que el asistente continúe desde allí. Por ejemplo, podrías comenzar el prompt con "Érase una vez, había una joven princesa que vivía en un castillo. Un día, recibió un regalo mágico que cambió su vida para siempre. ¿Cuál fue el regalo y cómo cambió su vida?" y el asistente podría generar una historia única y cautivadora basada en ese prompt.

Profesor:

Para crear un asistente que pueda enseñar a los usuarios sobre un tema específico, puedes diseñar prompts que proporcionen material de lección y hagan preguntas para evaluar la comprensión del usuario. Por ejemplo, podrías comenzar el prompt con "En esta lección, aprenderemos sobre las propiedades de la materia. ¿Cuáles son los tres estados de la materia?" y el asistente podría generar una respuesta que proporcione una explicación y un cuestionario para evaluar el conocimiento del usuario.

Al diseñar cuidadosamente los prompts y utilizar el poder de mistralai/mistral-

The content of this lab is licensed under Apache 2.0

proporcionar información valiosa a los usuarios. La clave es pensar en el tipo de salida que deseas que genere el asistente y diseñar los prompts en consecuencia. Solo queda crear una hermosa interfaz de usuario/experiencia de usuario alrededor de estos prompts y tendrás un negocio de un millón de dólares ;)!

Author(s)

Talha Siddiqui

Rohit Arora

Vicky Kuo

Contributor(s)

Kang Wang

© IBM Corporation. Todos los derechos reservados.

Pez Babel con LLM STT TTS



Skills Network

Tiempo estimado necesario: 90 minutos

Introducción

Bienvenido a este proyecto guiado sobre la creación de un Pez Babel. El Pez Babel es una metáfora para un servicio de traducción o una herramienta, basada en una criatura ficticia de la serie "La guía del autoestopista galáctico" de Douglas Adams.

En este proyecto, crearás un asistente de traducción por voz utilizando Watsonx y

The content of this lab is licensed under Apache 2.0

construir un asistente de traducción que puede recibir entrada de voz, convertirla a texto utilizando tecnología de reconocimiento de voz, enviar el texto al modelo mistralai/mistral-large de Watsonx, recibir una respuesta en otros idiomas, convertirla a voz utilizando tecnología de texto a voz y finalmente reproducirla al usuario. El traductor de voz tendrá un frontend responsivo utilizando HTML, CSS y JavaScript, y un backend confiable utilizando Flask.

Haz clic aquí para jugar con una demostración de la aplicación final que crearás.

Al final del curso, tendrás las habilidades para crear tu propio asistente de traducción impulsado por IA capaz de recibir entrada de voz y proporcionar traducciones en varios idiomas. También tendrás una base sólida en desarrollo web utilizando Python, Flask, HTML, CSS y JavaScript, junto con una aplicación de pila completa totalmente funcional que seguramente impresionará a cualquiera que interactúe con ella.

Antes de comenzar, proporcionemos un poco de contexto sobre cada tema.

Watsonx

Watsonx es una plataforma de IA y datos con un conjunto de asistentes de IA diseñados para ayudarte a escalar y acelerar el impacto de la IA con datos confiables en toda tu empresa.

Los componentes centrales incluyen un estudio para nuevos modelos fundamentales, IA generativa y aprendizaje automático; un almacén de datos adecuado construido sobre una arquitectura de lago de datos abierto; y un conjunto de herramientas, para acelerar flujos de trabajo de IA que se construyen con responsabilidad, transparencia y explicabilidad.

Los asistentes de IA de Watsonx empoderan a las personas en tu organización para realizar trabajos sin conocimientos especializados en una variedad de procesos y aplicaciones empresariales, incluyendo la automatización del servicio al cliente, la generación de código y la automatización de flujos de trabajo clave en departamentos como RRHH.

Bibliotecas de IBM Watson Speech para Embed

IBM Watson® Speech Libraries for Embed es un conjunto de bibliotecas de texto a voz y de voz a texto en contenedores diseñadas para ofrecer a nuestros socios de IBM una mayor flexibilidad para incorporar lo mejor de la tecnología de IBM

The content of this lab is licensed under Apache 2.0

obtienen mayores capacidades para construir aplicaciones de transcripción de voz y síntesis de voz más rápidamente y desplegarlas en cualquier entorno híbrido de múltiples nubes. Estas tecnologías permiten que el asistente se comunique con los usuarios a través de entrada y salida de voz.

Asistentes de voz

Un asistente virtual es un programa diseñado para simular la conversación con usuarios humanos, especialmente a través de Internet utilizando la voz humana natural. Los asistentes pueden ser utilizados en una variedad de industrias, incluyendo el servicio al cliente, el comercio electrónico y la educación.

Python (Flask)

Python es un lenguaje de programación popular que se utiliza ampliamente en el desarrollo web y la ciencia de datos. Flask es un marco web para Python que facilita la construcción de aplicaciones web. Utilizaremos Python y Flask para construir el backend de nuestro asistente de voz. Python es un lenguaje poderoso que es fácil de aprender y tiene un gran ecosistema de bibliotecas y marcos que se pueden aprovechar en proyectos como el nuestro.

HTML - CSS - JavaScript

HTML (Lenguaje de Marcado de Hipertexto) es un lenguaje de marcado utilizado para estructurar contenido en la web. CSS (Hojas de Estilo en Cascada) es un lenguaje de hojas de estilo utilizado para describir la apariencia y el formato de un documento escrito en HTML. JavaScript es un lenguaje de programación que se utiliza comúnmente para agregar interactividad a las páginas web. Juntas, estas tecnologías nos permiten construir un frontend visualmente atractivo e interactivo para nuestro asistente. Los usuarios podrán interactuar con el asistente de voz a través de una interfaz web construida utilizando HTML, CSS y JavaScript.

Objetivos de aprendizaje

Al final de este proyecto, serás capaz de:

- Explicar los fundamentos de los asistentes de voz y sus diversas aplicaciones.
- Configurar un entorno de desarrollo para construir un asistente utilizando Python, Flask, HTML, CSS y JavaScript

The content of this lab is licensed under Apache 2.0

- Implementar la funcionalidad de voz a texto para permitir que el asistente entienda la entrada de voz de los usuarios.
- Integrar el asistente con el modelo mistralai/mistral-large de Watsonx para dotarlo de un alto nivel de inteligencia y la capacidad de entender y responder a las solicitudes de los usuarios.
- Implementar la funcionalidad de texto a voz para permitir que el asistente se comunique con los usuarios a través de salida de voz.
- Combinar todos los componentes anteriores para crear un asistente funcional que pueda recibir entrada de voz y proporcionar una respuesta hablada.
- Desplegar como aplicación web ejecutando un servidor en modo de desarrollo.

Requisitos previos

Para trabajar en este proyecto, deberías tener un conocimiento fundamental de Python. Además, tener conocimientos básicos de HTML, CSS y JavaScript es recomendable, pero no esencial. Haremos nuestro mejor esfuerzo para explicar cada paso del proceso, así como cualquier código mostrado a lo largo del camino.

Paso 1: Comprendiendo l… →

The content of this lab is licensed under Apache 2.0